```
import pandas as pd
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from torch.nn.utils.rnn import pad_sequence
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

import torch
print(torch.cuda.is_available())
print(torch.cuda.device_count())
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else
"No GPU")
```

```
False
0
No GPU
```

```
# Device configuration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cpu
```

```
data = pd.read_csv(r"ner_dataset (1).csv", encoding="latin1").ffill()
words = list(data["Word"].unique())
tags = list(data["Tag"].unique())

if "ENDPAD" not in words:
    words.append("ENDPAD")

word2idx = {w: i + 1 for i, w in enumerate(words)}
tag2idx = {t: i for i, t in enumerate(tags)}
idx2tag = {i: t for t, i in tag2idx.items()}

data.head(5)
```

	Sentence #	Word	POS	Tag
0	Sentence: 1	Thousands	NNS	O
1	Sentence: 1	of	IN	O
2	Sentence: 1	demonstrators	NNS	O
3	Sentence: 1	have	VBP	O
4	Sentence: 1	marched	VBN	O

```
print("Unique words in corpus:", data['Word'].nunique())
print("Unique tags in corpus:", data['Tag'].nunique())
```

```
Unique words in corpus: 35177
Unique tags in corpus: 17
```

```
print("Unique tags are:", tags)
```

```
Unique tags are: ['O', 'B-geo', 'B-gpe', 'B-per', 'I-geo', 'B-org',
'I-org', 'B-tim', 'B-art', 'I-art', 'I-per', 'I-gpe', 'I-tim', 'B-
nat', 'B-eve', 'I-eve', 'I-nat']
```

```
# Group words by sentences
class SentenceGetter:
    def __init__(self, data):
        self.grouped = data.groupby("Sentence #",
group_keys=False).apply(
            lambda s: [(w, t) for w, t in zip(s["Word"], s["Tag"])]
        )
        self.sentences = list(self.grouped)

getter = SentenceGetter(data)
sentences = getter.sentences
```

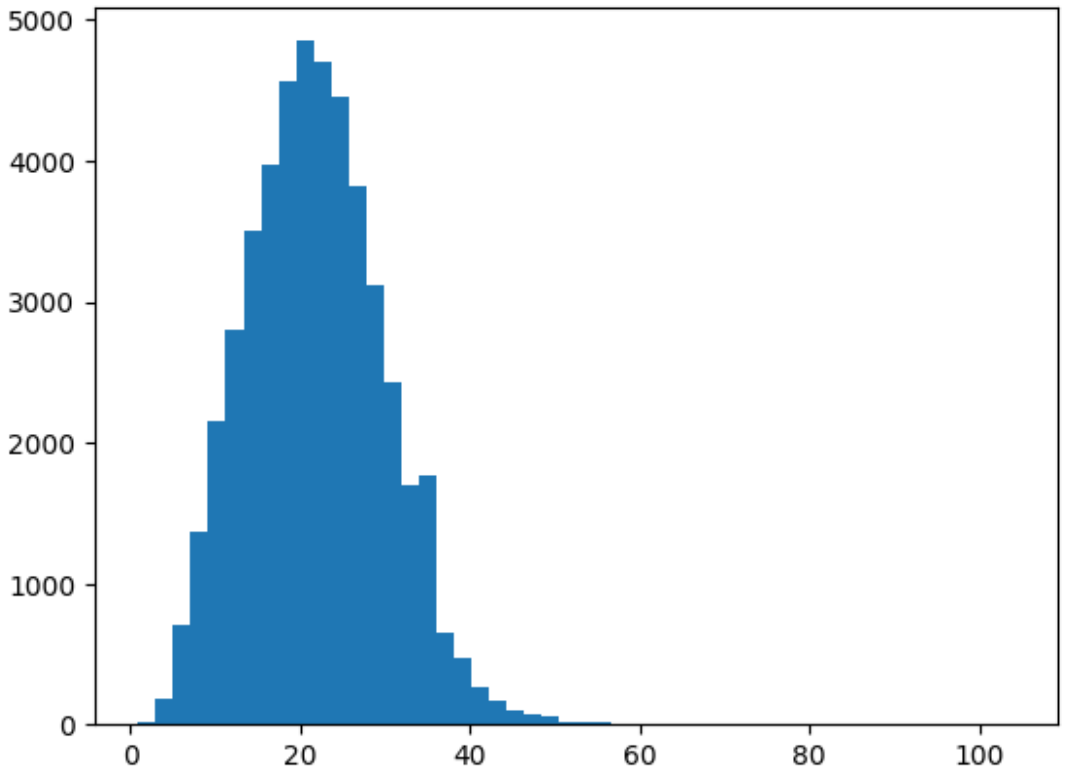
```
C:\Users\KISHORE\AppData\Local\Temp\ipykernel_19452\3613252132.py:4:
FutureWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  self.grouped = data.groupby("Sentence #", group_keys=False).apply(
```

```
sentences[35]
```

```
[('The', 'O'),
 ('U.S.', 'B-org'),
 ('Geological', 'I-org'),
 ('Survey', 'I-org'),
 ('gave', 'O'),
 ('a', 'O'),
 ('preliminary', 'O'),
 ('estimate', 'O'),
 ('of', 'O'),
 ('the', 'O'),
 ('strength', 'O'),
 ('of', 'O'),
 ('the', 'O'),
 ('Tuesday', 'B-tim'),
 ('morning', 'I-tim'),
 ('quake', 'O'),
 ('at', 'O'),
 ('6.7', 'O'),
```

```
 ('on', 'O'),
 ('the', 'O'),
 ('Richter', 'B-geo'),
 ('scale', 'O'),
 (',', 'O'),
 ('and', 'O'),
 ('said', 'O'),
 ('the', 'O'),
 ('epicenter', 'O'),
 ('was', 'O'),
 ('close', 'O'),
 ('to', 'O'),
 ('the', 'O'),
 ('island', 'O'),
 ('of', 'O'),
 ('Nias', 'B-org'),
 ('.', 'O')]
```

```
# Encode sentences
X = [[word2idx[w] for w, t in s] for s in sentences]
y = [[tag2idx[t] for w, t in s] for s in sentences]

plt.hist([len(s) for s in sentences], bins=50)
plt.show()
```



```
# Pad sequences
max_len = 50
X_pad = pad_sequence([torch.tensor(seq) for seq in X],
batch_first=True, padding_value=word2idx["ENDPAD"])
y_pad = pad_sequence([torch.tensor(seq) for seq in y],
batch_first=True, padding_value=tag2idx["O"])
X_pad = X_pad[:, :max_len]
y_pad = y_pad[:, :max_len]

X_pad[0]
```

```
tensor([    1,     2,     3,     4,     5,     6,     7,     8,     9,
10,
           11,    12,    13,    14,    15,    10,    16,     2,    17,
18,
           19,    20,    21,    22, 35178, 35178, 35178, 35178, 35178,
35178,
        35178, 35178, 35178, 35178, 35178, 35178, 35178, 35178, 35178,
35178,
        35178, 35178, 35178, 35178, 35178, 35178, 35178, 35178, 35178,
35178])
```

```
y_pad[0]
```

```
tensor([0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 2, 0, 0,
0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0,
        0, 0])
```

```
# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X_pad, y_pad,
test_size=0.2, random_state=1)

# Dataset class
class NERDataset(Dataset):
    def __init__(self, X, y):
        self.X = X
        self.y = y

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return {
            "input_ids": self.X[idx],
            "labels": self.y[idx]
        }

train_loader = DataLoader(NERDataset(X_train, y_train), batch_size=32,
```

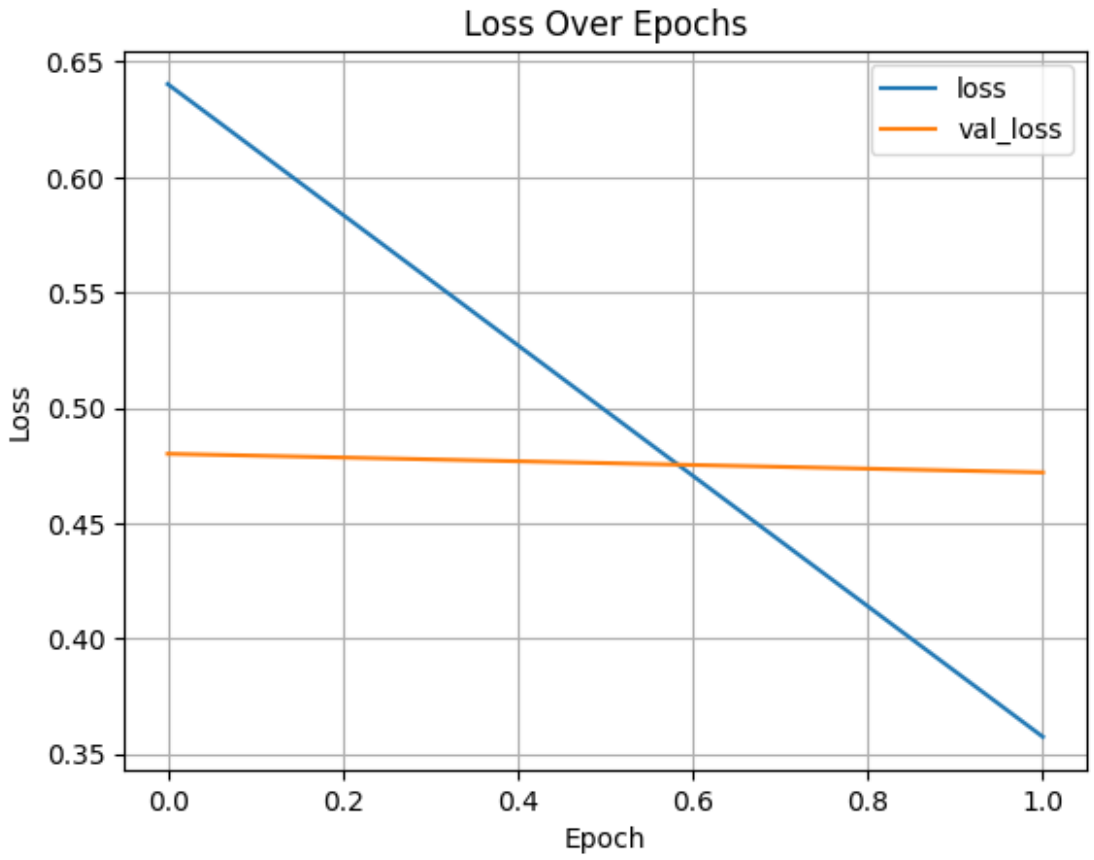
```
shuffle=True)
test_loader = DataLoader(NERDataset(X_test, y_test), batch_size=32)


# Model definition
class BiLSTMTagger(nn.Module):
    def __init__(self, vocab_size, num_tags, embedding_dim=50,
hidden_dim=50):
        super(BiLSTMTagger, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim,
padding_idx=vocab_size-1)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim,
batch_first=True, bidirectional=True)
        self.fc = nn.Linear(hidden_dim * 2, num_tags)

    def forward(self, input_ids):
        embeds = self.embedding(input_ids)
        lstm_out, _ = self.lstm(embeds)
        tag_space = self.fc(lstm_out)
        return tag_space

model = BiLSTMTagger(vocab_size=len(word2idx) + 1,
num_tags=len(tag2idx)).to(device)
loss_fn = nn.CrossEntropyLoss(ignore_index=tag2idx["O"])
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

# Training and Evaluation Functions
def train_model(model, train_loader, test_loader, loss_fn, optimizer,
epochs=2):
    train_losses, val_losses = [], []
    for epoch in range(epochs):
        model.train()
        total_loss = 0
        for batch in train_loader:
            input_ids = batch["input_ids"].to(device)
            labels = batch["labels"].to(device)

            optimizer.zero_grad()
            outputs = model(input_ids)
            outputs = outputs.view(-1, outputs.shape[-1])
            labels = labels.view(-1)

            loss = loss_fn(outputs, labels)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        train_losses.append(total_loss / len(train_loader))

        model.eval()
```

```
        val_loss = 0
        with torch.no_grad():
            for batch in test_loader:
                input_ids = batch["input_ids"].to(device)
                labels = batch["labels"].to(device)
                outputs = model(input_ids)
                outputs = outputs.view(-1, outputs.shape[-1])
                labels = labels.view(-1)
                loss = loss_fn(outputs, labels)
                val_loss += loss.item()

        val_losses.append(val_loss / len(test_loader))
        print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_losses[-
1]:.4f} - Val Loss: {val_losses[-1]:.4f}")

    return train_losses, val_losses

def evaluate_model(model, test_loader, X_test, y_test):
    model.eval()
    true_tags, pred_tags = [], []
    with torch.no_grad():
        for batch in test_loader:
            input_ids = batch["input_ids"].to(device)
            labels = batch["labels"].to(device)
            outputs = model(input_ids)
            preds = torch.argmax(outputs, dim=-1)
            for i in range(len(labels)):
                for j in range(len(labels[i])):
                    if labels[i][j] != tag2idx["O"]:
                        true_tags.append(idx2tag[labels[i][j].item()])
                        pred_tags.append(idx2tag[preds[i][j].item()])

# Run training and evaluation
train_losses, val_losses = train_model(model, train_loader,
test_loader, loss_fn, optimizer, epochs=2)
evaluate_model(model, test_loader, X_test, y_test)
```

```
Epoch 1/2 - Train Loss: 0.6403 - Val Loss: 0.4801
Epoch 2/2 - Train Loss: 0.3574 - Val Loss: 0.4720
```

```
# Plot loss
print('Name: SARAVANAN P ')
print('Register Number:212224230253')
history_df = pd.DataFrame({"loss": train_losses, "val_loss":
val_losses})
history_df.plot(title="Loss Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

```
Name: SARAVANAN P
Register Number:212224230253
```




```python
# Inference and prediction
i = 125
model.eval()
sample = X_test[i].unsqueeze(0).to(device)
output = model(sample)
preds = torch.argmax(output, dim=-1).squeeze().cpu().numpy()
true = y_test[i].numpy()

print('Name: SARAVANAN P ')
print('Register Number:212224230253')
print("{:<15} {:<10} {}\n{}".format("Word", "True", "Pred", "-" * 40))
for w_id, true_tag, pred_tag in zip(X_test[i], y_test[i], preds):
    if w_id.item() != word2idx["ENDPAD"]:
        word = words[w_id.item() - 1]
        true_label = tags[true_tag.item()]
        pred_label = tags[pred_tag]
        print(f"{word:<15} {true_label:<10} {pred_label}")
```

```
Name: SARAVANAN P
Register Number:212224230253
Word            True       Pred
-----------------------------------------
Palestinian     B-gpe      B-gpe
officials       O          B-gpe
say             O          B-tim
two             O          B-tim
Palestinians    B-gpe      B-gpe
have            O          B-tim
been            O          I-tim
killed          O          B-geo
in              O          B-tim
an              O          B-tim
accidental      O          B-org
explosion       O          B-gpe
in              O          B-tim
a               O          B-tim
West            B-org      B-org
Bank            I-org      I-org
refugee         O          I-org
camp            O          B-per
.               O          I-org
```